CS2600+CS2610 — Computer Organization and Architecture (+ lab)

Tier	A
Phase (order)	3
Do this first	CS2300 (or at least gates + assembly idea)
Estimated effort	12 weeks
Unlocks	CS3500, CS4100, CS6600

Roadmap

1. Instruction set architectures, addressing modes, function calls, stack frames (RISC-V/ARM/x86).
2. Single-cycle datapath and control; then pipelining and hazards.
3. Caches and memory hierarchy; virtual memory.
4. Superscalar, multithreaded, multicore basics.
5. Lab: assembly programs, multi-precision arithmetic, calling C from assembly, and a simulator for caches/branch predictors.

Video courses (all links checked on YouTube on 2026-10-22)

🎓 Main course: follow it start to end

- Computer Sc - Computer Organization (33 videos). NPTEL *Computer Organization* (nptelhrd, IIT Madras).

🎥 Other full lecture series (pick one if the main one doesn't click, or for topics it skips)

- Livestream - Digital Design and Computer Architecture (Spring 2026) (51 videos). Onur Mutlu *Digital Design and Computer Architecture* (ETH Zurich, Spring 2026 livestream). World-class; do the labs.
- computer Organization (76 videos). IIT Madras BS *Computer Organization*.
- Computer Architecture and Organisation (63 videos). NPTEL *Computer Architecture and Organisation* (IIT Kharagpur): very thorough on pipelines and memory.
- [CS61C FA20] Lectures - Great Ideas in Computer Architecture | UC, Berkeley (100+ videos). Berkeley CS61C Fall 2020 (re-upload by a viewer, not the official channel): C + RISC-V + caches + parallelism.

Study plan (week by week)

Lecture numbers are positions in the main playlist above. Click a number to open that video.

Week	Lectures	What to do
1-2	1–5	Computing systems overview, processor activities, processor as a state machine.
3-4	6–14	Datapath, microprogrammed vs hardwired control, addressing modes. Write small RISC-V assembly programs (RARS simulator).
5-6	15–18	Memory system and caches. Simulate a direct-mapped cache in Python.
7	19–23	Virtual memory, segmentation, address translation.
8-9	24–33	I/O: programmed, interrupt-driven, DMA, buses.
10-12	Mutlu ETH (alt)	Pipelining, hazards, branch prediction, out-of-order. Build a 5-stage pipelined RISC-V CPU in Verilog (lab).

▼ **All lectures in *Computer Sc - Computer Organization* (33 videos)**

1. Lecture - 1 Introduction To Computing
2. Lecture - 2 Introduction To System : Software
3. Lecture - 3 Introduction To System : Hardware
4. Lecture - 4 Processor Activities
5. Lecture - 5 Processor As a State Machine
6. Lecture - 6 Data Path Architecture
7. Lecture - 7 Data Path Controller : Micro Programmed
8. Lecture - 8 State Machine Design
9. Lecture - 9 Controller Design: Microprogrammed and Hardwired
10. Lecture - 10 Controller Design (Contd)
11. Lecture - 11 Typical Micro Instructions
12. Lecture - 12 Addressing Modes
13. Lecture - 13 Problem Exercise
14. Lecture - 14 Problem Exercise
15. Lecture 15 - Inroduction to memory system
16. Lecture - 16 CPU - Memory Interaction
17. Lecture - 17 Cache Organization
18. Lecture - 18 Cache Organization
19. Lecture - 19 Virtual Memory
20. Lecture - 20 Virtual Memory
21. Lecture - 21 Performance Calculation
22. Lecture - 22 Segmentation
23. Lecture - 23 Address Translation and Protection
24. Lecture - 24 Programmed I/O
25. Lecture 25 - Interrupt Driven I/O
26. Lecture - 26 DMA : Direct Memory Access
27. Lecture - 27 Device Service Routines

28. Lecture - 28 Evolution Of I/O
29. Lecture 29 - I/O devices
30. Lecture 30 - I/O Devices - Contd
31. Lecture 31 - Buses
32. Lecture 32 - Buses Contd
33. Lecture - 33 Conclusion

Books, notes and websites

- ✅ NPTEL *Computer Organization and Architecture* (Kamakoti, IIT Madras) — nptel.ac.in/courses/106106166
- Book: Patterson & Hennessy, *Computer Organization and Design, RISC-V Edition*
- ETH Zurich *Digital Design and Computer Architecture* (Onur Mutlu) — YouTube
- Berkeley CS61C lectures; Ripes simulator for RISC-V
- Book: Bryant & O'Hallaron, *CS:APP*

Practice / proof of understanding

Write a recursive factorial and a string-reverse in RISC-V assembly, trace a pipeline with hazards by hand, and run a cache simulator comparing associativities.

What your IITM course sheet says

From the PDF: CS2600

Official description: To learn about the internal organization of CPU and physical memory unit, different types of Instruction Set Architecture (ISA) used in CPUs, ISA support for programming languages and operating system functions, cache and memory hierarchy, architecture of scalar pipelined processor, and introduction to superscalar, multi-threaded, and multi-core processors.

Official syllabus (trimmed):

1. Introduction to Structured Computer Organization: Digital hardware level, Microarchitecture level, Operating system level, System programming level and Application programming level. Differences between computer organization and computer architecture.
2. Instruction Set Architecture Support for Programming Languages: Arithmetic, logical, and comparison instructions, Addressing modes, Data movement instructions, Control transfer instructions, Instruction representation, Instruction decoding, Control unit, Atomic instructions, Stack frames, Function calls and parameter passing, and CISC and RISC architectures.
3. CPU Subsystem Architecture: Design of a single cycle processor for basic instructions learnt in the previous module, CPU System-on-Chip (SoC) architecture with core and memory mapped peripherals, Interrupt as a means for peripherals to communicate with the core, Different types of interrupts, and Platform-level vs. core-level interrupt controller. This module acts as a bridge between the earlier module on instruction set and the subsequent module on ISA support for operating systems.
4. Instruction Set Architecture Support for Operating Systems: ISA support for handling interrupts, Software interrupts, Nested interrupts, Using interrupts to support multi-tasking and the resulting challenges, Virtual memory, Demand paging, Page table, Hierarchical page tr

Official textbooks: 1. J.L.Hennessy and D.A.Patterson, Computer Architecture -- A Quantitative Approach, 6th Ed., Morgan Kaufmann, 2017. 2. J.P.Shen and M.H.Lipasti, Modern Processor Design -- Fundamentals of Superscalar Processors, Tata McGraw-Hill, 2005 3. W.Stallings, Computer Organization and Architecture – Designing for Performance, 9th Edition, Pearson, 2013.

Official references: 1. C.Hamacher, Z.Vranesic, S.Zaky and N.Manjikian, Computer Organization and Embedded Systems, 6th Edition, McGraw-Hill, 2012.

From the PDF: CS2610

Official description: The primary objective of this course is to teach assembly language programming using the RISC-V or ARM or x86 Instruction Set Architecture (ISA), its support for translation of high-level programming language constructs and data structures into assembly programs, and implementation of operating system related functions using assembly programs. The secondary objective is to introduce students to custom-built or open-source architectural simulators for analyzing the performance of pipelined processors, cache hierarchy, and branch predictors.

Official syllabus (trimmed):

1. Set up and understand the ISA simulator/emulator toolchain, compile sample assembly programs, and simulate the executables using both normal and debug modes of operation.
2. Write assembly programs to perform arithmetic, logical, and data transfer operations, implement loops for array traversal, and use branch instructions for flow control.
3. Write assembly program to perform multi-precision arithmetic operations. Such operations are used in cryptographic algorithms like RSA that perform computations on 1024-bit integers or deep learning algorithms that use IEEE full-precision/half-precision floating point or bfloat16 format (developed by Google).
4. Write recursive programs in assembly language to understand argument passing, stack maintenance, and local variable allocation.
5. Understand how to access functions and variables defined in C from assembly programs and vice versa, through proper use of assembler directives and calling conventions.
6. Learn how to switch privilege modes by programming the appropriate control and status registers.
7. Simulate a simple cooperative multitasking environment in assembly language by implementing a program with two concurrently running tasks and a basic context switching mechanism triggered by timer interrupts.
8. Understand virtual memory management by switching between different privilege modes, setting up page tables

Official textbooks: 1. Sivarama P. Dandamudi, Introduction to Assembly Language Programming for Pentium and RISC Processors, Springer-Verlag, 2005. 2. Daniel Kusswurm, Modern x86 Assembly Language Programming: 32-bit, 64-bit, SSE, and AVX, Apress, 2014. 3. D.A. Patterson and J.L. Hennessy. Computer Organization and Design: The Hardware/Software Interface (RISC-V Edition). Morgan Kaufmann Publishers, 2020.

Official references: 1. Igor Zhirkov, Low-Level Programming: C, Assembly, and Program Execution on Intel® 64 Architecture, Apress, 2017.